

Advanced PUF Protocol: A Machine Learning Resilient PUF-based Protocol for Lightweight Mutual Authentication

Tarek Idriss, Haytham Idriss, Magdy Bayoumi, *Fellow IEEE*

The Center for Advanced Computer Studies
University of Louisiana at Lafayette
Lafayette, Louisiana, USA

Abstract— PUFs or Physical Unclonable Functions are emerging hardware security primitives that can offer a lightweight security solution for constrained devices in IoT through challenge-response authentication protocols. PUFs, however, were found to be vulnerable to modeling attacks. This work introduces a lightweight PUF-based protocol for mutual authentication that is resilient against known modeling attacks. The protocol utilizes several techniques to introduce non-linearity. The protocol can utilize any Strong PUF circuit for which a soft model can be generated. The authentication process requires a simple PUF circuit and a true random number generator (TRNG) and avoids the use of any cryptographic or hash functions to preserve the lightweight nature of the protocol. The security of the protocol against modeling attacks is evaluated and shown to have an exceptional resilience figure. The protocol high resilience against modeling attack would renew confidence in Strong PUFs as authentication primitives that can tackle emerging IoT security threats. Similar PUF-based protocols are investigated and found to have significant security vulnerabilities and other shortcomings when compared to the introduced protocol.

Index Terms—hardware security, security protocol, lightweight security, Physical Unclonable Functions, cryptography.

1 INTRODUCTION

Establishing reliable security for constrained devices has been an ongoing challenge due to the high constraints on power consumption, implementation area or device cost. The security of the authentication process in some of these small devices is of critical importance. With the emergence of the Internet of Things (IoT) the security of such devices have become one of the major issues facing modern computing. Standard cryptographic solutions that provide provable security have prohibitive area and power demands for many applications like RFIDs, medical implants, or smart cards. For instance, low cost RFID tags can only use 3-5K logic gates for security functions as reported in [1] and [2] while public cryptography algorithms implementations which are crucial for reliable key exchange [3][4] can use between 12k to 22k logic gates [5].

Silicon based Physical Unclonable Functions or PUFs [6] are emerging hardware security primitives that have the potential to offer security solutions for constrained devices due to their low implementation overhead. PUFs exploit the inherent randomness within the manufacturing process of CMOS devices to produce a unique response when offered an input ‘challenge’. As the PUF response depends on the randomness of the miniscule variations within the manufacturing process, PUFs are inherently unique and ‘unclonable’. This makes them ideal for establishing secure challenge-response protocols.

The challenge-response space of a PUF determines whether it is classified as a weak PUF or a strong PUF [7]. Weak PUFs are characterized by having a relatively small challenge-

response. Such PUFs are mostly used as an alternative to secure key storage. Strong PUFs, on the other hand, have a large challenge-response space making them useful for authentication protocols.

Although unclonable through hardware, strong PUFs have been found to be vulnerable to modeling attacks in [8], [9] and [10]. In such attacks, adversaries would collect the exchanged challenge-response pairs (CRPs) used in authentication sessions and apply machine learning algorithms to produce a software model of the PUF circuit. Such model would be capable of correctly predicting the response of any incoming PUF input challenge. Consequently, PUF-based authentication protocols had to either employ very large PUF circuits to ensure resilience against modeling attacks or use cryptographic primitives to fully secure the challenge-response exchange such as in [11], [12] and [13]. However, either solutions would introduce significant implementation overhead making it prohibitive for many low power applications and would ultimately defeat the purpose of utilizing PUFs. As an alternative to the computationally expensive standard cryptographic algorithms, PUF-based authentication protocols that use simple pseudo-cryptographic algorithms were suggested in [14], [15] and [16]. However, such protocols would offer less reliable security or limit the number of possible authentications to ensure resilience against modeling attacks.

In this paper, we introduce a secure, lightweight PUF-based authentication protocol that can offer an unlimited number of mutual authentications. The Advanced PUF Protocol (APP) hides the PUF challenge-response correlations by conditionally

transforming the challenges utilized. The protocol exchanges challenges that are generated by a true random number generator (TRNG) and then authenticates parties by checking for a required correlation between the exchanged challenges. The correlations under test are hidden using conditional transformation functions that transform the transmitted challenges dynamically prior to using them as inputs to the PUF circuit. The protocol’s resilience against machine learning attacks is established by testing the performance of known attacks on the protocol where we show its superior resilience nature.

We briefly list the main contribution of this work:

- We introduce a lightweight PUF-based mutual authentication and secret message exchange protocol based on a novel challenge-challenge exchange that can hide both the PUF responses and challenges. The protocol security and exceptional resilience against machine learning attacks is demonstrated by performing such attacks on a simulated setup.
- We introduce novel challenge transformation functions that can transform the highly correlated challenges of Arbiter-based PUFs into multiple unique uncorrelated challenges while requiring a small implementation overhead.
- We present a security analysis of the protocol and compare its security with other recently introduced lightweight PUF-based authentication protocols to highlight the introduced protocol advantages.

The remainder of this paper is organized as follows. Section 2 is a background on PUFs and PUF-based authentication protocols. Section 3 introduces the challenge-challenge exchange and the Advanced PUF Protocol. Section 4 introduces the design of the uncorrelated challenge transformation function that can be used to address the naturally high correlations of Arbiter-based PUF challenges. A detailed security analysis of the protocol is performed in Section 5 where we test the protocol’s resilience against machine learning attacks. In Section 6, the error tolerance of the protocol is illustrated. The introduced protocol features and security are compared with related work in Section 7. Section 8 concludes this paper.

2 BACKGROUND

2.1 Delay Arbiter PUF

We examine the Delay Arbiter PUF [6] as it is one of the earliest and most studied silicon based PUFs. It is also a very simple PUF that can easily illustrate how minuscule process variation can be used to build non-clonable circuits. The Arbiter PUF also serves as a building block to other more complex strong PUF designs. The arbiter PUF compares the delay of two identical paths to generate either a ‘0’ or a ‘1’ bit depending on the result of the comparison. Although the two paths are identical and should introduce the same delay, unpredictable minuscule variations during the fabrication process ensures that one path is ultimately faster than the other.

Multiplexers, referred to as “switch-components”, are inserted into the paths. Challenge bits are used as select inputs to the multiplexers. Each switch-component either introduces crossed paths or straight paths depending on the select bit of the

multiplexer. This results in a high combination of possible paths. Figure 1 shows the architecture of such PUF.

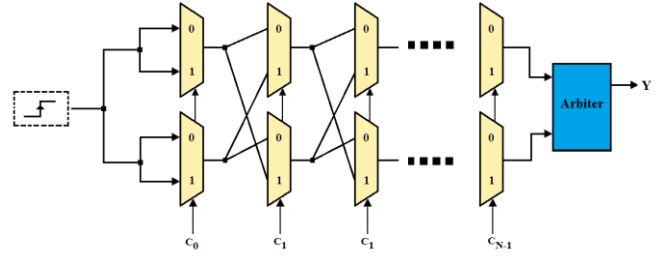


Figure 1: Delay Arbiter PUF Architecture

2.2 Modeling Attacks and Arbiter PUF Variations

The Arbiter PUF was found to be vulnerable to modeling attacks in [8], [9] and [10]. In such attacks, an adversary would collect the exchanged challenge-response pairs (CRPs) used in authentication sessions and apply machine learning algorithms to produce a software model of the PUF that is capable of correctly predicting the response of new challenges.

The Arbiter PUF shown in Fig. 1 can be modeled as a set of delay elements. The delay difference at the arbiter Δ can be expressed as a function of the differential delay vector ω , and Φ the feature vector that is a function of the input challenge [8]:

$$\Delta = \omega^T \Phi \quad (1)$$

Various machine learning algorithms can be used to determine the separating hyperplane $\omega^T \Phi = 0$ that serves as the decision boundary surface for the response bit. Linear Regression (LR) was shown to be a very efficient algorithm in solving for ω . For a 64-stage Arbiter PUF, observing 640 challenge-response pairs (CRPs) would allow an adversary to produce a soft model of the PUF with 95% accuracy and very little training time (< 1 sec).

To address this vulnerability, several modifications of the Arbiter PUF have been suggested. The XOR-Arbiter PUF [17] combines several rows of the simple arbiter PUFs by xor-ing their output into a single bit and introducing non-linearity in the PUF model. Other enhanced designs like the LS PUF [18] was suggested to increase the PUF resistance to modeling attacks as well. However, these introduced designs were also found to be vulnerable, albeit to a lesser extent, to modeling attacks in [8].

For highly non-linear PUFs like FF-PUF [19] machine learning techniques utilizing Evolution Strategies (ES) were utilized to successfully produce a soft model of the PUF [18]. ES generates multiple ‘offspring’ of an initial model, which can be described by a delay vector ω , using random mutations. The relative ‘fitness’ of the offspring models are evaluated and the fittest ones are used as new parents for the next generation. Evolution Strategy algorithms are particularly useful when the PUF challenge-response correlation is highly convoluted and linear regression would be infeasible to use. As long as the attacker can have a good measure of the relative fitness of the randomly generated models, the ES algorithm would eventually converge to the desired model.

It was suggested in [8] that modeling-resilient PUF designs would be possible to implement by drastically increasing the number of XORed delay elements in some PUF designs. However, such PUFs have been shown in [20] to require a large implementation area ranging from 22k to 30k GE making them

infeasible to implement in constrained devices. With this vulnerability to modeling attacks, PUF-based authentication protocols had to employ other techniques to hide the challenge-response correlations of utilized PUF circuits.

2.3 PUFs in Authentication Protocols

Figure 2 shows a ‘barebone’ PUF-based authentication protocol. A server with access to a soft model of the PUF would generate a set of challenges C and send it to the device. The device would use its PUF circuit to generate a set of responses R for the challenges in C and would send the responses back to the server. The server would compare the device’s responses with the ones generated from the soft model. If the received and generated responses are within certain error tolerance α , then the device would be deemed authentic.

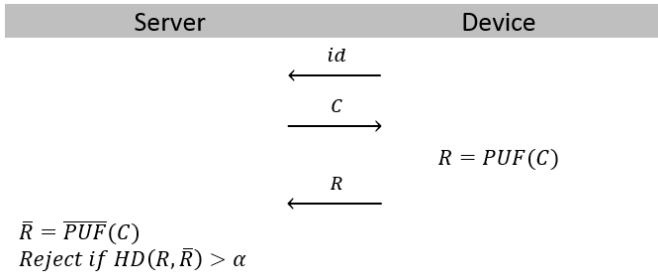


Figure 2: A ‘barebone’ PUF based challenge-response authentication protocol

The barebone authentication protocol described would be insecure as an adversary would be able to easily perform a modeling attack by collecting the exposed CRPs. Prior PUF-based protocols used varying methods to prevent such attacks. In [11], [12] and [13] cryptographic/hash functions were used to hide the challenge-response exchange, providing reliable security albeit at the cost of using expensive hardware implementations that might not be suitable for constrained devices and hence defeats the purpose of PUF-based security.

To avoid the use of such expensive hardware, PUF-based authentication protocols that utilize simple pseudo-cryptographic algorithms were introduced in [14], [15] and [16]. These protocols, however, offer less reliable security or limit the number of possible authentication. In a later section, the security limitations of such protocols are examined in detail. The introduced protocol would eliminate all such disadvantages as it was designed with resistance to machine learning techniques like ES in mind.

3 THE ADVANCED PUF PROTOCOL

3.1 Protocol Setup and Adversary Model

The protocol utilizes a Strong PUF circuit at the device side and a soft model of the PUF circuit at the server side. The Strong PUF circuit is treated as a black box by the protocol and hence any Strong PUF circuit can be used as long as it has an associated PUF model. The protocol would utilize a novel challenge-challenge exchange that can authenticate the queried party. We note that Weak PUF circuits would not be suitable as they have a small challenge-space. Figure 3 shows an illustration of how the protocol can be deployed device side and server side. On the

server side, the protocol would be implemented in software. When the unconstrained device cannot be trusted with permanent access, the PUF model and the protocol logic can be stored on a remote trusted server while unconstrained devices with temporary access can forward authentication requests to the trusted remote server. This would allow system administrators to remove the access rights of devices by simply revoking the access to the remote model and logic.

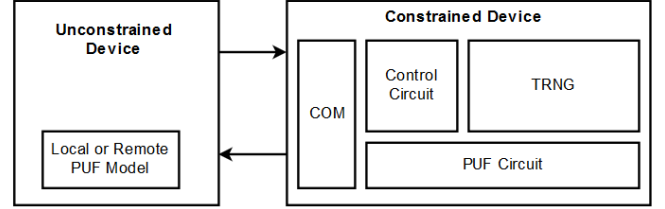


Figure 3: Overall APP system architecture

The adversary is assumed to have access to the communication channel used by the prover and verifier. The adversary can eavesdrop on the communication and may also perform substitution attacks. Protecting against probing attacks or side channel attacks are outside the scope of this work as the protocol’s purpose is to secure the communication between the verifier and the prover in the presence of an eavesdropper.

3.2 Description of Used Notations

- C_i (ex: $C_1, C_2 \dots$): indicates a single pseudo challenge that can be used to generate an input challenge by applying a transformation function on it. We note that the challenge-challenge exchange utilizes a single pseudo challenge C_1 as the verifier’s pseudo-challenge, and two pseudo challenges C_2 and C_3 as the prover’s response challenges.
- T_i (ex: $T_1, T_2 \dots$): indicates a transformation function that transforms a single pseudo-challenge C_j into a transformed input challenge $T_i(C_j)$.
- $PUF(T_i(C_j))$: indicates the single-bit output/response of the PUF circuit when presented with a transformed challenge $T_i(C_j)$. Only transformed challenges are fed to the PUF circuit.
- R_{Vi} (ex: $R_{V1}, R_{V2}, \dots$): indicates a single PUF response bit, when presented with the transformed verifying challenge $T_i(C_1)$.
- R_V (ex: 000, 001 ...): indicates the concatenation of three response bits R_{V1}, R_{V2} and R_{V3} . In every exchange, three transformations T_1, T_2 and T_3 are applied on the pseudo challenge C_1 to produce R_V .

3.3 Challenge Exchange Overall Description

The introduced novel challenge-challenge exchange allows two parties to engage in an authentication without exposing any of the PUF circuit responses while ensuring resistance against machine learning attacks. Figure 4 shows how two parties, each with access to a PUF circuit or its soft model, can perform authentication while only exchanging random ‘pseudo’ challenges. The exchanged challenges are dubbed as ‘pseudo’ challenges as they are never used as a direct challenge input to

the PUF circuit but rather as a seed to new challenges that would be dynamically generated. A random pseudo challenge $C1$ is presented by the verifier. The prover would receive the pseudo-challenge and would generate two random pseudo challenges $C2$ and $C3$ such that:

$$PUF(T_i(C2)) \oplus PUF(T_j(C3)) = g \quad (2)$$

Where T_i and T_j are challenge transformation functions that transform a pseudo-challenge to a unique input challenge. The response values of the challenges generated from $C1$ would decide which of the transformations are used on the prover pseudo-challenges. The choice of the transformations applied is hidden from the adversary, hence the actual PUF input is also hidden. The value of g is either '0' or '1' and is randomly selected by the prover at the start of each authentication round. The transformation steps are discussed in detail in the next subsection.

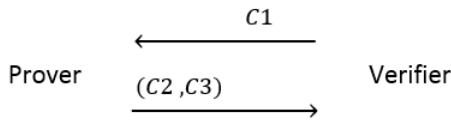


Figure 4: The challenge-challenge exchange can authenticate parties by exchanging randomly generated pseudo-challenges.

3.4 Combined Verifier-Prover Challenge Generation

Figure 6 shows how both the verifier and the prover generate a portion of the verifying challenge $C1$. For an n -bit challenge, each of the prover and verifier would generate $(n/2)$ -bit challenges and exchange them. The verifying challenge would simply be the concatenation of the generated challenges.

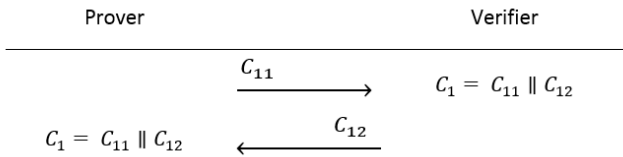


Figure 6: Verifying challenge $C1$ generated by both the prover and verifier using concatenation.

This guarantees the freshness of each challenge exchange as it prevents adversaries from polling the device using the same set of challenges. This, however, reduces the effective challenge space of the PUF to $(n/2)$ bits as half of the bits can now be controlled by the prover. To maintain an effective challenge space of n -bits, the protocol would need to utilize a $2n$ -bit input PUF.

3.5 Conditional Challenge Transformation

Figure 5 shows the flowchart for the conditional transformation utilized. The verifier pseudo challenge is transformed into three uncorrelated challenges, each producing a unique response. This would result in a 3-bit response for the verifier's pseudo-challenge. This response would be completely hidden from an attacker's perspective.

At the core of the protocol security is the conditional transformation of the challenges. This would hide the actual challenge-input of the PUF circuit. For this reason, we refer to the challenges $C1$, $C2$ and $C3$ as 'pseudo' challenges as they are never used as a direct input to the PUF circuit. The purpose of

the transformation is to hide the PUF circuit input and greatly increase the non-linearity of the system. In the discussed version of the protocol, we utilize four uncorrelated transformation functions, $T1$, $T2$, $T3$ and $T4$, to transform the pseudo-challenges exchanged into new uncorrelated challenges.

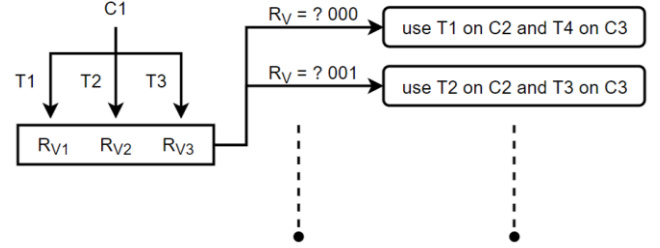


Figure 5: Prover challenges $C2$ and $C3$ are transformed conditionally based on the verifying challenge $C1$ transformed responses.

Table 1 shows the eight authentication conditions that could possibly be under test by the verifier. The eight unique conditions are a result of applying a unique combination of transformations on $C2$ and $C3$. From an observer perspective, any of these eight authentication conditions could have been under test. The choice of the transformations applied, and ultimately the authentication condition, is determined by the value of the responses of the verifier pseudo challenge $C1$.

TABLE 1: THE POSSIBLE CONDITIONS TESTED ACCORDING TO THE VALUES OF THE RESPONSES GENERATED FROM THE VERIFYING PSEUDO-CHALLENGE $C1$

R_V			Authentication Condition
R_{V1}	R_{V2}	R_{V3}	$PUF(T_i(C2)) \oplus PUF(T_j(C3)) = g$
0	0	0	$PUF(T1(C2)) \oplus PUF(T4(C3)) = g$
0	0	1	$PUF(T2(C2)) \oplus PUF(T3(C3)) = g$
0	1	0	$PUF(T3(C2)) \oplus PUF(T2(C3)) = g$
0	1	1	$PUF(T4(C2)) \oplus PUF(T1(C3)) = g$
1	0	0	$PUF(T1(C2)) \oplus PUF(T3(C3)) = g$
1	0	1	$PUF(T2(C2)) \oplus PUF(T4(C3)) = g$
1	1	0	$PUF(T3(C2)) \oplus PUF(T1(C3)) = g$
1	1	1	$PUF(T4(C2)) \oplus PUF(T2(C3)) = g$

As g is a random value chosen by the prover, the verifier would have no access its value. To test the authenticity of the prover, the verifier would simply check whether the pseudo-challenges presented by the prover in an authentication session would correspond to a single value of g . An adversary impersonating an authentic party would fail to produce a consistent value of g for the all the m exchanges required as a proof of identity. The randomness of g adds little computational overhead to the protocol while it significantly increases its security against machine learning based attacks. This is discussed in more detail in a later subsection where we demonstrate the protocol security against such attacks.

The design of four transformation functions that can guarantee generating four uncorrelated challenges from a single pseudo-challenge can be a non-trivial task when utilizing some PUF circuits. In an ideal PUF, all challenge inputs would be completely uncorrelated. This would make the transformation function a very trivial challenge expansion function that simply

uses the pseudo-challenge as a seed to generate three unique challenges. However in practice some PUFs like the Arbiter PUF and its variants have high correlation among their challenge inputs. In this work we introduce a novel method for producing uncorrelated challenges when using the Arbiter PUF and its variants like the XOR PUF, LS PUF or FF PUF. We note though that using such PUFs is not required by the protocol, we simply introduce this novel method to illustrate how such PUFs with a naturally high correlation between their input challenges can still be used in our protocol with very little overhead.

3.6 Authentication Protocol Steps

Figure 7 shows the authentication protocol utilizing the transformed challenges exchange. The verifier and prover would be parties in possession of the PUF circuit or an accurate soft model of the PUF which can be used to generate responses for any random challenge. As the verification procedure require lightweight computation, both a PUF device or a standard device with a PUF soft model can authenticate the other party. This is one of the major advantages of the proposed protocol as other protocols like Slender [14] or [16] require computationally heavy verification algorithms that are unsuitable to small devices and thus can only support one-way device authentication.

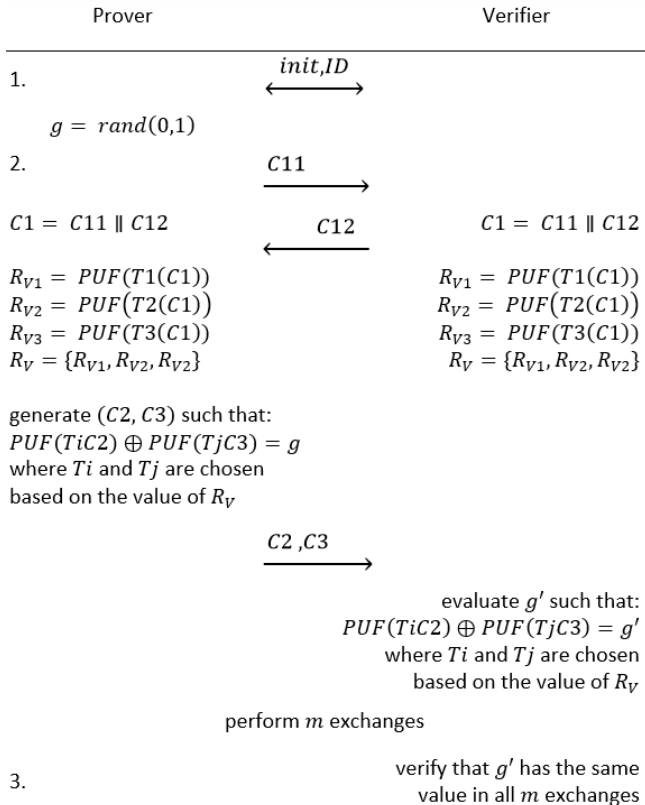


Figure 7: The detailed steps of the authentication protocol utilizing the novel challenge-challenge exchange.

In step 1 of the protocol an authentication session initialization message and the PUF ID are exchanged. The prover would generate a random value for g (0 or 1) that would be used for this session. In step 2 of the protocol the transformed challenge exchange is performed m times. As the verifier has no access to the value of g , they would simply save the value of

$\text{PUF}(Ti(C1)) \oplus \text{PUF}(Tj(C3))$ in g' . In step 3 of the protocol, the verifier would check whether the value of g' is the same in all the m exchanges performed. If not, it would reject to authenticate the device. The protocol can be modified to allow for error tolerance by allowing for mismatches in a small portion of the generated g' values. Such tolerance rate should be calibrated in accordance with the utilized PUF's circuit error rate.

An adversary would be unable to produce a fixed value of g for all the m exchanges in the authentication session. The probability for randomly generating such pseudo-challenges would be equal to $2 \times 2^{-m} = 2^{-(m-1)}$, which is equivalent to the probability of randomly producing a g value of 0 for all m exchanges or a g value of 1 for all m exchanges. This guarantees an exponential level of security against random guess attacks. We further investigate the security of the protocol against modeling attacks and challenge collection attacks in section 5 of this paper.

3.7 Secret Message Exchange

TABLE 2: POSSIBLE RESULTS OF SECRET DATA BIT EXCHANGE

$C2, C3$			Authenticity (for $g = 0$)	Data bit d_i
$\text{PUF}(Ti(C2))$	$\text{PUF}(Tj(C3))$	$\oplus$		
0	0	0	Authentic	0
0	1	1	Rejected	N/A
1	0	1	Rejected	N/A
1	1	0	Authentic	1

The enhanced challenge-challenge exchange can support secret message exchange. By having two possible authentic response values for a fixed value of g , we can encode each response with a different data bit. An adversary could only guess the actual data bit as the response of a PUF challenge is never exposed. Table 2 shows how the two possible authentic combinations can each be encoded with a different data bit in a session where $g=0$.

4 DESIGN OF UNCORRELATED CHALLENGE TRANSFORMATION FUNCTIONS

In an ideal PUF, all the challenge inputs would be completely uncorrelated. This would make a challenge transformation function very simple. Each transformation function would add a unique bit string to the pseudo-challenge resulting in a unique challenge associated with the transformation and the pseudo-challenge used. However in practice some PUFs like the Arbiter PUF and its variants have high correlation among their challenge inputs and as such unique challenges are not necessarily uncorrelated [18]. This was one of the issues addressed in the design of the LS PUF [18].

In this work we introduce a novel method for producing uncorrelated challenges when using the Arbiter PUF and its variants like the XOR PUF, LS PUF, FF PUF or DA-PUF [21]. We note though that using such PUFs is not required by the protocol, we simply introduce this novel method to illustrate how these PUFs with a naturally high correlation between their input challenges can still be used in our protocol with very little overhead. In this section, we first examine the impact of induced

challenge bit-flips on the response of an Arbiter PUF. We then explain how multiple input challenge bit-flips can be used to design uncorrelated challenge transformation functions.

4.1 Input Challenge Bit-Flips Response Correlation

The authors in [22] examined the effect of introducing a single bit-flip in the PUF input challenge on the output response of an arbiter PUF. We verified and reproduced their work using an arbiter PUF mathematical model. We used the same delay arbiter PUF model utilized in [22] which are based on the delay variations presented in [23]. The results are shown in Fig. 8 where we can see that input bit-flips introduced near the middle of the PUF circuit have a $\sim 50\%$ chance of producing a bit-flip at the PUF output.

Prior to this work, the effect of multiple induced challenge bit-flips on Arbiter PUFs have not been properly examined. We have observed that introducing multiple bit-flips would have a drastically different effect than that of a single bit-flip. For instance, when using the same mathematical model to produce Fig. 8, introducing two bit-flips at locations 31 and 33 respectively, would result in a 10% probability of obtaining a response bit-flip. This very low probability is contrary to the intuition of figure 8, where both bit-flips showed a probability very close to 50%.

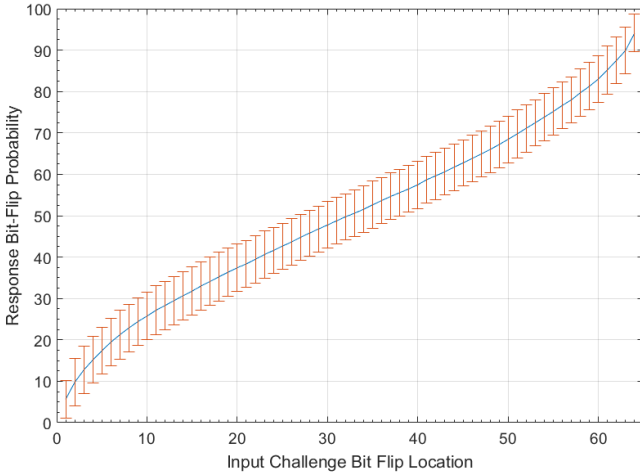


Figure 8: Probability of response bit-flip as a result of single input-challenge bit-flip of a 64-bit Arbiter PUF.

To understand the PUF output correlations when multiple bit-flips are introduced we examine a case study where five instances of input bit-flips are introduced into a 64-bit PUF circuit. Figure 9 shows the effect and location of the introduced bit-flips instances A, B, C, D and E on a 64-bit Arbiter PUF circuit.

The shaded areas in each instance show the portion of the PUF where the delay elements have swapped paths in the circuit (upper vs lower) due to the introduced challenge bit-flips. Delay elements are either connected to the same path prior to the induced bit-flips (non-shaded) or swapped their path as a result of the induced bit-flips (shaded). We conjecture that the probability P_s of two instances of induced bit-flips to share the same PUF's response value can be predicted by the ratio of matching delay elements connections over the total number of delay elements. When the shaded delay elements ratio is around

50%, the probability of producing a matching response should be around 50%.

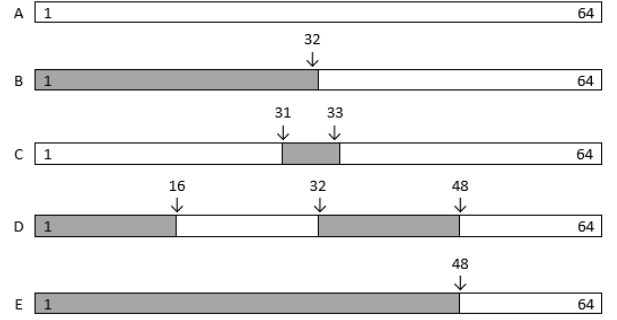


Figure 9: The tested bit-flip instances and their effect on the delay elements path connections. Shaded areas represent delay elements that have swapped paths.

To verify our conjectured estimation method, we used the mathematical model of the Arbiter PUF to evaluate the output of random challenges when the bit-flips instances shown in figure 9 are utilized. A total of 200 PUF instances were produced in and 5000 challenges tested in each instance. Table 3 shows the experimental results of the bit-flips instances along with the ratio of shared delay elements.

The results show that indeed the probability of introducing a response bit-flip is quasi-proportional to the ratio of shared delay elements. It can be seen that a 50% mismatch ratio would always yields a 50% probability of sharing the same response. For ratio values different than 50%, the relationship seems to be quasi-proportional rather than proportional. The full relationship between the ratio of matching connection and output response is shown in Figure 10. The data was produced by introducing multiple bit-flips at random locations and examining the response correlation with various challenges.

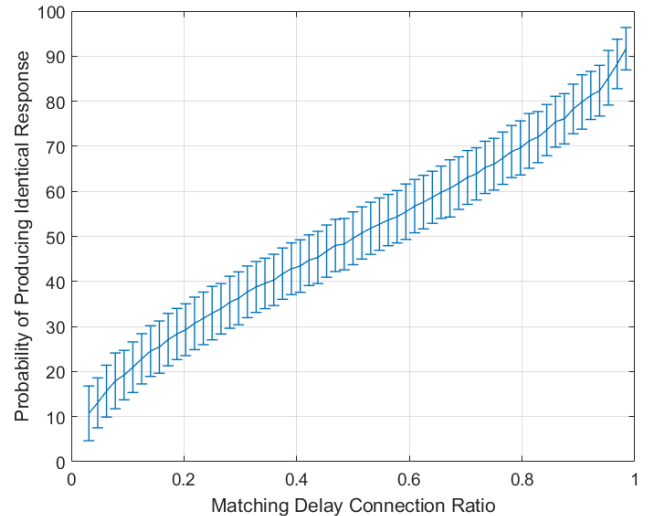


Figure 10: Probability of response bit-flip correlation with the ratio of matching delay elements.

TABLE 3: THE TESTED INSTANCES MATCHING CONNECTION RATIO AND THE ASSOCIATED PROBABILITY OF PRODUCING THE SAME OUTPUT RESPONSE BIT.

	Correlations with instance A			
	AB	AC	AD	AE
Mismatch Ratio	0.50	0.95	0.5	0.25

Probability P_S	0.505	0.885	0.501	0.338
-------------------	-------	-------	-------	-------

From the figure we can see that an uncorrelated response can always be produced by introducing challenge bit-flips that results in a 0.5 ratio of mismatched delay elements connections. This insight is used in the next subsection to design four uncorrelated transformation functions.

4.2 Design of Uncorrelated Transformation Function

Figure 11 illustrates how the padding bits chosen for the transformation functions would result in four unique circuit connections that each have a matching delay element connection (shaded/unshaded) ratio of 0.5 when compared to any other transformation instance. This ensures that there would be no correlation between any of the transformed challenges. For a protocol utilizing 128-bit pseudo challenges and a 131-bit arbiter PUF circuit, these locations would be 34, 65, and 97.

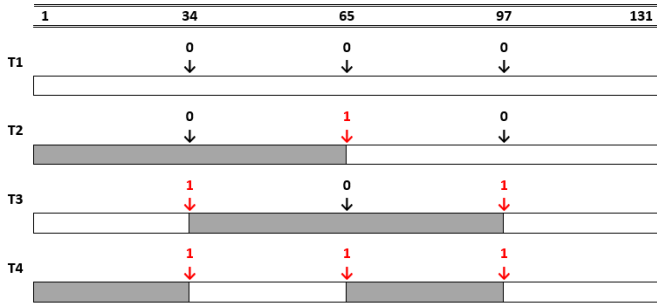


Figure 11: The transformation functions T1-T4 are designed by inserting additional stages in the Arbiter PUF that are used to insert bit-flips at sensitive location that ensure

The correlation among the transformation functions was verified using the Arbiter PUF mathematical model. The result of the verification is shown in Table 4, where 200 Arbiter PUF instances were fed with 5000 random pseudo-challenges. We can see that the functions are uncorrelated as the probability of producing the same response would be very close to 0.5.

TABLE 4: THE TRANSFORMATION FUNCTIONS CROSS CORRELATION PROBABILITIES PRODUCED STATISTICALLY FROM THE PUF MODEL.

Transformation	T2	T3	T4
T1	$\mu=0.499$ $\sigma=0.042$	$\mu=0.487$ $\sigma=0.046$	$\mu=0.498$ $\sigma=0.039$
T2		$\mu=0.497$ $\sigma=0.038$	$\mu=0.486$ $\sigma=0.044$
T3			$\mu=0.498$ $\sigma=0.041$

5 PROTOCOL SECURITY ANALYSIS

We analyze the security of the Advanced PUF-based Protocol for Lightweight Mutual Authentication against various attacks. We assume the protocol employs an Arbiter PUF with an n -bit input challenge and utilizes m exchanges in each authentication session. For simplicity, we ignore the error margin α as it can be addressed by increasing the number of exchanges m as we show in a later section that examines the effect of the PUF error rate on the protocol security.

5.1 Random Guessing

In a random guess attack, the attacker simply responds with randomly generated pseudo-challenges. The probability of success for a random guess attack would be $2^{-(m-1)}$. A relatively small value of m would be sufficient for most applications as the attack does not expose any information or leak the model of the PUF.

5.2 Modeling Attacks

As the protocol hides both the challenges utilized and the PUF circuit response, it would protect the PUF circuit from modeling attacks that require direct access to the challenge-response pairs of the circuit. For each exchange, an attacker would have to guess which of the eight possible transformations occurred. The number of possible guesses required would grow exponentially when trying to perform such attacks. Machine learning attacks that utilize linear regression would fall into this category as they require a direct correlation between the circuit input and output.

However, machine learning attacks utilizing Evolution Strategies (ES) algorithm can be performed without such direct access to the challenge response pairs. ES would generate random models and test their ‘fitness’ according to the collected data. As long as the attacker can perform a ‘fitness’ evaluation for a random model, such attacks can still be performed. In [26] ES was used to successfully generate an accurate model of the PUF circuits utilized by Slender. To evaluate the security of the proposed protocol against ES, we performed a Covariance Matrix Adaptation (CMA-ES) using a local implementation of the algorithm presented in [27].

We also performed the CMA-ES attack on Slender [16] to evaluate the advantage and the relative security gains of our introduced protocol. Slender protocol hides the response of the PUF circuit in ‘plain sight’ by concatenating the PUF responses with a random bit stream generated by a TRNG and then logically rotating the string by random number to hide the index of the authentic response substring. The server receiving the full string would then perform a substring matching algorithm to determine whether the authentic substring is part of the device response. The introduced protocol showed a solid resilience to CMA-ES attacks even when utilizing the notoriously vulnerable Arbiter PUF as the choice of the PUF circuit. We note how the CMA-ES was quickly able to compromise the PUF circuit model utilized by Slender.

We utilized a local implementation of the CMA-ES algorithm presented in [27]. The fitness function used in the attack against Slender measured the alignment score of the generated string and the collected string. This was done by rotating the collected strings and measuring the hamming distance from the tested model’s generated string after each rotation. The normalized average of the minimum hamming distance among the strings under test was used as a final fitness value. Such fitness evaluation would give a very good measure of the relative models accuracy in the pool and this allows the ES algorithm to converge confidently toward more accurate models.

In the attack against our introduced protocol, the fitness function would evaluate the performance of the random PUF models by measuring consistency of the g value in each round. A bias towards either values of $g=0$ or $g=1$ would correspond to a better fitness score. Here we see how the randomness of the g

value in each session plays a critical role in confusing the ES algorithm particularly when the model is around 50% accuracy. While an authentic round might have a g value of '0', a bias towards a g value of '1' would also give the model a better fitness score although the tested model would have less accuracy.

TABLE 5: NUMBER OF CHALLENGES AND OBSERVED AUTHENTICATIONS UTILIZED IN THE CMA-ES ATTACKS ON SLENDER AND APP.

	Total Pool Size		Iteration Sample Size	
	# of chall.	# of auth.	# of chall.	# of auth.
Slender	1.25 million	1000	125,000	100
APP	3 million	10,000	200,000	2000

In the ES attacks on the simulated setups, APP utilizes a 131-bit Arbiter PUF circuit and a number of $m=100$ exchanges per authentication. Slender utilized a 131-bit Arbiter PUF and a full string length of $L=2500$ and a substring of $L_s=1250$ which is a similar setup to the one used in [16]. The simple Arbiter PUF is notorious for its low security and vulnerability, yet it was selected for testing purposes as its vulnerability would allow for a more noticeable impact of the protocol security rather than the PUF circuit security. For the attack on Slender, we utilized a pool of 1.25 million challenges which formed 1000 observed authentications. The ES algorithm randomly selected 100 observations, equivalent to 125,000 challenges in each iteration to evaluate the fitness function. For the attack on the introduced Advanced PUF Protocol (APP), we utilized 1 million observed exchanges, equivalent to the observation of 3 million pseudo challenges or 10,000 authentication sessions when using $m=100$ exchanges per authentication session. The ES would select 2000 random authentication sessions containing 200,000 exchanges for each iteration to evaluate the fitness function. Table 5 summarizes these values.

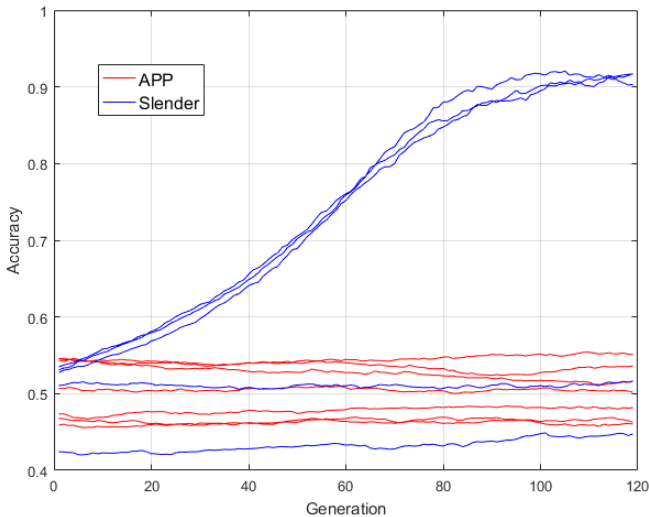


Figure 12: Accuracy levels achieved by the CMA-ES attacks on Slender (blue) and on the introduced APP (red).

Figure 12 shows the achieved model accuracy progression of several runs over 120 generations of ES attacks on Slender (blue) and the proposed APP (red). We can see clearly how most ES attacks quickly converges on an accurate PUF model when utilizing Slender while all the attacks on APP (red) failed to make

any noticeable accuracy gains on APP even for runs initialized with relatively high accuracy. The normalized fitness function values and their progression are shown in figure 13. The figure also shows how the CMA-ES algorithm was very effective in minimizing the fitness value F on Slender (blue) while the fitness value of APP (red) did not show a noticeable decrease.

Overall, the results showed that the introduced protocol is vastly superior to protocols like Slender when it comes to its resilience against machine learning attacks. Although the attacks failed to successfully model the Arbiter PUF circuit employed, we do not claim that the introduced protocol would be immune to such attacks when using a vulnerable PUF like the Arbiter PUF utilized in our experiment. Due to the low memory of the Arbiter PUF and its relatively low uniqueness a dedicated attacker with access to high computational capacities might eventually generate an accurate model of the PUF with enough restarts. The Arbiter PUF was chosen for testing to have a better measure of the security gains of the protocol when utilizing more vulnerable PUFs. The experiment did clearly indicate the highly superior resilience the introduced protocol would offer. For reliable security, we would recommend the use of a 3-row XOR PUF or LS PUF when considering Arbiter-based PUFs. Any Strong PUFs with similar security can also be utilized by the protocol as there is no restriction to the Strong PUF circuit besides the availability of a soft model.

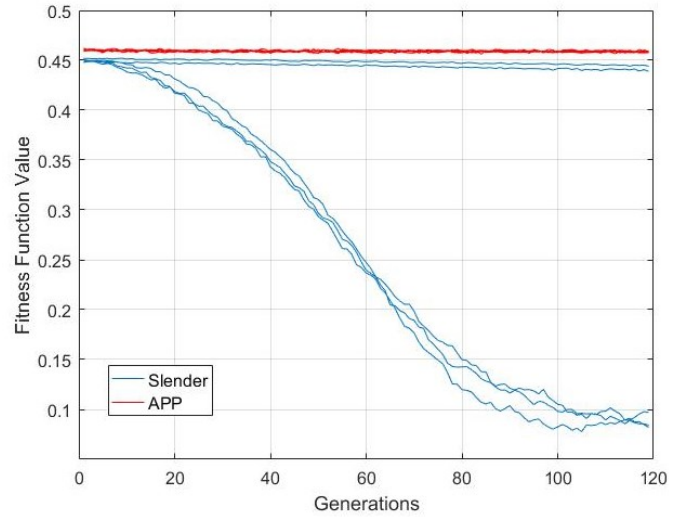


Figure 13: Normalized fitness function values over 120 generations for attacks on Slender (blue) and the introduced APP (red). A lower value would indicate a better matching model.

To the best of our knowledge, the introduced protocol would be the first to protocol to show such high resilience against ES-based machine learning attacks while employing no cryptographic primitives and while also avoiding any limitations on the number of authentication attempts supported like Lockdown protocol in [15]. The greater impact of the results is that lightweight authentication protocols utilizing strong PUFs are more than viable and are far from being a pipe dream as they can be resilient to machine learning attacks when properly implemented even when using delay arbiter PUFs variants like the XOR PUF or LS PUF. The high effectiveness of machine learning attacks on strong PUFs have made many question their usability and the introduced Advanced PUF Protocol would

renew confidence in the role Strong PUFs can play in tackling emerging security threats.

5.3 Exhaustive Challenge Collection Attack

By observing and storing the authentic exchange containing the pseudo challenges $\{C1, C2, C3\}$, the attacker can produce an authentic response $\{C2, C3\}$ when presented with the same pseudo-challenge $C1$ by the verifier. An adversary would be able to use stored authentic exchanges to enhance their random guess attack efficiency. Figure 14 shows how the protocol security level against random guess attacks (y-axis) varies with respect to the number of authentic exchanges collected by the adversary (x-axis) for a challenge input bit-width of $n=64$ (red) and $n=128$ (blue). The figure shows that there would be no significant security loss until a significant portion the challenges is collected.

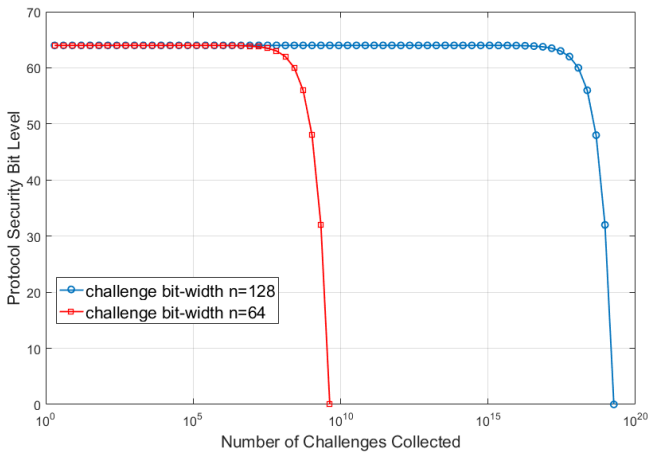


Figure 14: The effect of the number of authentic pseudo-challenges collected by an adversary on the protocol bit-level security against random guess attacks.

The PUF is fully compromised when the attacker has collected an authentic challenge response $\{C2, C3\}$ for every possible challenge $C1$ that can be produced by the verifier. Since the prover controls half of the n bits of the challenge $C1$, this would require the observation of $2^{n/2}$ challenge exchanges and the storage of $2^{n/2} \times 3 \times n = 2^{0.5n + \log_2(3n)}$ bits making the attack completely infeasible to perform for a sufficiently large value of n as the protocol would guarantee a $0.5n$ -bit security level against it. In our version of protocol, we utilize $n = 128$ to guarantee 64-bit security level.

While 64-bit cryptographic security is obsolete in modern day technology, it is extremely secure against challenge collection attacks as the challenge collection cannot be parallelized or accelerated when it is limited to the speed of the associated device. Furthermore, the challenges observed need to be stored which is infeasible. For instance, for $n=128$ the adversary would need to store more than 800 exabytes of data.

6 PUF CIRCUIT ERROR RATE AND TUNING

Due to the noisy outputs of PUF circuits, the authentication protocol would require employing some error tolerance value α . Such error tolerance would depend on the expected number of non-authentic challenges produced by an authentic node due to the natural circuit error rate of the PUF. A simple MATLAB

binary search algorithm script was used along with the inverse binomial cumulative distribution function, also known as the quantile function, to find the minimum value of m that can maintain the desired random guess success probability while supporting the required error tolerance rate.

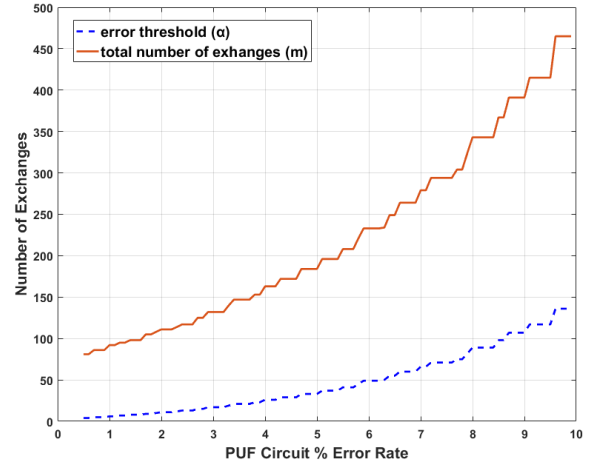


Figure 15: The required growth of m and α to compensate for the PUF circuit error rate while maintaining 64-bit level of security.

Figure 15 shows how the values of m and α increase to compensate for the PUF circuit error while maintaining 64-bit security level. The fault tolerance value α is chosen to be 3% higher than the value of the expected error rate of non-authentic challenges E_{NA} . We note that the error rate of non-authentic challenges is higher than the PUF circuit error rate as multiple PUF challenge evaluations are involved in each authentication.

Table 6 highlights the tuning values of α and m at some key values of the PUF error rate. At each key point, the table shows the value of the PUF circuit error rate E_C , the expected non-authentic challenges rate in the protocol session E_{NA} , the tolerance rate t , the number of exchanges m that need to be utilized to maintain 64-bit security, and the number of allowed non-authentic exchanges α .

TABLE 6: NUMBER OF EXCHANGES REQUIRED FOR 64-BIT SECURITY AT KEY ERROR RATES VALUES.

% PUF Circuit Error	% Faulty Exchanges	% Fault Tolerance	# Exchanges m	Fault Tolerance Threshold α
$E_C = 0.5\%$	$E_{NA} = 1.7\%$	$t = 4.7\%$	$m = 81$	$\alpha = 4$
$E_C = 4\%$	$E_{NA} = 12.6\%$	$t = 15.6\%$	$m = 163$	$\alpha = 26$
$E_C = 8\%$	$E_{NA} = 22.5\%$	$t = 25.5\%$	$m = 343$	$\alpha = 89$

The table shows that the value of m and α remain manageable even when utilizing circuits with up to 8% error rate. The values can be tuned dynamically depending on the operating conditions and the aging effect of the circuit. We note though that when using the Advanced PUF protocol for secret message exchange, higher error rate can be problematic even when the authenticity of the message can be confirmed. As such, it is advisable to use a highly reliable PUF in addition to error correction techniques when deploying the protocol for secret message exchange.

7 RELATED WORK COMPARISON

There have been several attempts on lightweight PUF-based authentication protocols that avoid the use of cryptographic functions like encryption or hashing. However, the protocols introduced so far suffer from various disadvantages or vulnerabilities when compared to the Advanced PUF Protocol. The authors in [24] did a comprehensive survey on various PUF based protocols. We compare our work with the most successful lightweight protocols from their work along with recently introduced protocols that includes the Lockdown protocol [15] and challenge-obfuscation [16] protocol.

7.1 Lockdown Protocol

Lockdown Protocol [15] limits the number of possible authentication attempts a device can perform. This ensures that a PUF would never reveal enough challenge-response pairs to allow an adversary to model the PUF circuit. The protocol employs a counter at the server side to keep track of the number of authentication attempts performed. Such solution is very simplistic and ineffective as the protocol fails to address the main challenge associated with it, which would be securing the protocol against denial of service (DoS) attacks that aim to exhaust the supported number of authentications. The authors did recognize the DoS threat and suggested that the server involved in the authentication should employ a DoS detection mechanism. However, no explanations or details of such mechanism nor its effectiveness have been provided. DoS attacks are very easy to perform and there is no solid security measure that can completely stop these attacks. By simply jamming the communication channel or impersonating a device an adversary can quickly exhaust the limited number of supported authentications. The Lockdown protocol would be as such vulnerable to DoS attacks that exhaust its limited number of supported authentications rendering the PUF device unusable.

7.2 Slender Protocol

Slender protocol introduced in [14] is similar to the Advanced PUF-based Protocol when it comes to the building blocks used: a TRNG, a PUF circuit and a small control circuit. Slender protocol hides the response of the PUF circuit in ‘plain sight’ by concatenating the PUF responses with a random bit stream generated by a TRNG and then logically rotating the string by random number to hide the index of the authentic response substring. The server receiving the full string would then perform a substring matching algorithm to determine whether the authentic substring is part of the device response. The protocol however is vulnerable to a man in the middle attacks that can expose the location of the substring index or corrupt its secret key exchange. Furthermore, it fails to offer mutual authentication as the substring matching algorithm is too computationally expensive for constrained devices. In [26] it was shown that ES algorithm can successfully produce an accurate model of the PUF circuit employed. In this work, we introduce a man in the middle attack (MITM) that can further reduce the security of the protocol by exposing the location of the authentic index in a linear number of attempts. To the best of our knowledge, this attack on the Slender protocol has not been revealed in prior literature.

Figure 16 shows how a man in the middle can substitute several bits in the prover response with their inverted values. The substitutions target two portions of the string that are $(L_a - q)$ bits apart, where L_a is the length of the authentic substring and q the number of bit-flips introduced. The adversary introduces just enough bit-flips to ensure exceeding the error tolerance of the protocol. If the substituted bits end up being part of the authentic substring, then the device authentication would fail.

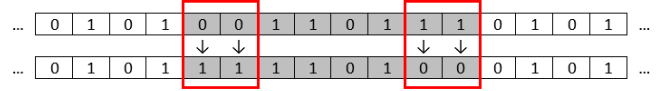


Figure 16: MITM substitution attack performed on the Slender protocol. The attacker targets two blocks of bits that are $L_a - q$ bits apart in an attempt to cause an authentication failure.

When the adversary observes an authentication failure after performing this substitution attack, they can conclude that the substituted bits were part of the authentic substring, otherwise the authentication would have succeeded. Since the two substituted portions are $(L_a - q)$ bits apart, the adversary can also conclude that the substitutions occurred at the boundaries of the authentic substring. This would reveal the location of the authentic substring with a high accuracy and allows the adversary easier access to the challenge-response pairs. The probability P_S for such attack to succeed would be equal to the probability of choosing a random index that happens to be the index of the authentic substring starts. Given that L_S is the length of the full string, P_S would be defined as:

$$P_S = \frac{1}{L_S} \quad (9)$$

We can see that the protocol can only offer linear security against this attack as the probability of success is proportional to $1/L_S$. To achieve a bit level security of 64-bits, L_S would need to have a value of 2^{64} which is infeasible. In the version of the protocol that is used for secret key exchange, the index of the authentic substring would be the secret message exchange. This secret key exchange is vulnerable to a MITM attack where the adversary rotates the transmitted bit string and alter the index value while preserving the authenticity of the message.

7.3 Other Protocols and Summary

Another recently introduced lightweight authentication method is the Obfuscated Challenge-Response [16]. However, The Obfuscated Challenge-Response protocol does not offer mutual authentication and would require a large circuit of XOR-PUFs to achieve a solid security level. In [25] the Noise Bifurcation Protocol showed increased resistance against modeling attacks, however, it does not claim a guaranteed level of security and it's also unable to offer mutual authentication. Table 7 compares the Advanced PUF Protocol (APP) with the other mentioned lightweight PUF-based protocols.

The introduced protocol shows several advantages over the other lightweight protocols. Neither *slender* [12] nor *noise bifurcation* [17] nor *obfuscated challenge-response* protocols offer mutual authentication. All three suffer from increased vulnerability to modeling attacks as well when compared to the Advanced PUF-based Protocol. While *lockdown IIb* [13] does provide mutual authentication, it can only support a limited number of authentications which makes it also vulnerable to DoS

attacks. The introduced Advanced PUF Protocol (APP) has none of these shortcomings and its high resilience against machine learning attacks like ES would put it in a different league when compared to known protocols that do not utilize cryptographic primitives. Furthermore, the protocol would be the first to allow for authenticated secret message exchange between all parties.

TABLE 7: LIGHTWEIGHT PUF-BASED SECURITY PROTOCOLS FEATURES AND COMPARISON

Protocol	TRNG Required	Number of Auth.	Modeling Robust	Mutual Auth.	Secret Mess.
<i>Slender</i>	✓	∞	~	×	~
<i>Noise bifur</i>	✓	∞	~	×	×
<i>Lockdown IIb</i>	✓	d	✓	✓	×
<i>Obfuscated Ch.</i>	✓	∞	~	×	×
<i>APP (this work)</i>	✓	∞	✓	✓	✓

8 CONCLUSION

We introduced a secure and robust PUF-based protocol that offers unlimited number of mutual authentications and secret message exchanges while utilizing a lightweight implementation that requires no cryptographic or hash functions. We also introduced a novel challenge transformation method that addresses the high correlation between challenges for Arbiter-based PUFs. The protocol's high resiliency against machine learning attacks is demonstrated by testing the performance of the CMA-ES attack on observed authentications. The introduced protocol security and features are compared with similar PUF-based protocols and found to be superior as we expose some security vulnerabilities of the similar protocols. The protocol high resilience against machine learning attacks renews confidence in Strong PUFs and the role they can play in tackling rising threats facing constrained devices in IoT.

REFERENCES

- [1] Chien, Hung-Yu. "SASI: A new ultralightweight RFID authentication protocol providing strong authentication and strong integrity." *IEEE Transactions on Dependable and Secure Computing* 4.4 (2007): 337-340.
- [2] Sarma SE. "Towards the five-cent tag." Technical Report MIT-AUTOID-WH-006, MIT Auto ID Center, 2001. Available at: <http://www.autoidlabs.org> [2001].
- [3] Boyko, Victor, Philip MacKenzie, and Sarvar Patel. "Provably secure password-authenticated key exchange using Diffie-Hellman." In *International Conference on the Theory and Applications of Cryptographic Techniques*, pp. 156-171. Springer, Berlin, Heidelberg, 2000.
- [4] Bellare, Mihir, David Pointcheval, and Phillip Rogaway. "Authenticated key exchange secure against dictionary attacks." In *international conference on the theory and applications of cryptographic techniques*, pp. 139-155. Springer, Berlin, Heidelberg, 2000.
- [5] Liu, Dongsheng, et al. "Design and Implementation of An ECC-Based Digital Baseband Controller for RFID Tag Chip." *IEEE Transactions on Industrial Electronics* 62.7 (2015): 4365-4373.
- [6] Gassend, B., et al., "Silicon physical random functions." *Proceedings of the 9th ACM conference on Computer and communications security*. ACM, 2002.
- [7] Guajardo, J., et al. "FPGA intrinsic PUFs and their use for IP protection." *International workshop on Cryptographic Hardware and Embedded Systems*. Springer Berlin Heidelberg, 2007.
- [8] Rührmair, U., et al., "PUF modeling attacks on simulated and silicon data." *IEEE Transactions on Information Forensics and Security* 8.11 (2013): 1876-1891.
- [9] Tobisch, J., & Georg T. B., "On the Scaling of Machine Learning Attacks on PUFs with Application to Noise Bifurcation." *International Workshop on Radio Frequency Identification: Security and Privacy Issues*. Springer International Publishing, 2015.
- [10] Ganji, F., Shahin T., & Seifert, J.P., "Why attackers win: on the learnability of XOR arbiter PUFs." *International Conference on Trust and Trustworthy Computing*. Springer International Publishing, 2015.
- [11] Gassend, B., Clarke, D., Van Dijk, M., & Devadas, S. (2002). "Controlled Physical Random Functions." *Security with Noisy Data*. Springer London, 2007. 235-253.
- [12] Sadeghi, Ahmad-Reza, Ivan Visconti, and Christian Wachsmann. "PUF-enhanced RFID security and privacy." In *Workshop on Secure Component and System Identification (SECSI)*. 2010.
- [13] Van Herrewwege, Anthony, et al. "Reverse fuzzy extractors: Enabling lightweight mutual authentication for PUF-enabled RFIDs." *International Conference on Financial Cryptography and Data Security*. Springer Berlin Heidelberg, 2012.
- [14] Majzoobi, Mehrdad, et al. "Slender PUF protocol: A lightweight, robust, and secure authentication by substring matching." *Security and Privacy Workshops (SPW), 2012 IEEE Symposium on*. IEEE, 2012.
- [15] Yu, Meng-Day Mandel, et al. "A Lockdown Technique to Prevent Machine Learning on PUFs for Lightweight Authentication." *IEEE Transactions on Multi-Scale Computing Systems* 2.3 (2016): 146-159.
- [16] Gao, Yansong, Gefei Li, Hua Ma, Said F. Al-Sarawi, Omid Kavehei, Derek Abbott, and Damith C. Ranasinghe. "Obfuscated challenge-response: A secure lightweight authentication mechanism for PUF-based pervasive devices." In *Pervasive Computing and Communication Workshops (PerCom Workshops), 2016 IEEE International Conference on*, pp. 1-6. IEEE, 2016.
- [17] Suh, G. E., & Srinivas D., "Physical unclonable functions for device authentication and secret key generation." *Proceedings of the 44th annual Design Automation Conference*. ACM, 2007.
- [18] Majzoobi, M., Farinaz K., & Miodrag P., "Lightweight secure pufs." *Proceedings of the 2008 IEEE/ACM International Conference on Computer-Aided Design*. IEEE Press, 2008.
- [19] Lee, J. W., et al., "A technique to build a secret key in integrated circuits for identification and authentication applications." *VLSI Circuits, 2004. Digest of Technical Papers. 2004 Symposium on*. IEEE, 2004.
- [20] T. Idriss, H. Idriss and M. Bayoumi, "A PUF-based paradigm for IoT security," *2016 IEEE 3rd World Forum on Internet of Things (WF-IoT)*, Reston, VA, 2016, pp. 700-705. doi: 10.1109/WF-IoT.2016.7845456
- [21] H. Idriss, T. Idriss and M. Bayoumi, "A highly reliable dual-arbiter PUF for lightweight authentication protocols," *2017 IEEE International Conference on RFID Technology & Application (RFID-TA)*, Warsaw, 2017, pp. 248-253. doi: 10.1109/RFID-TA.2017.8098903

- [22] Majzoobi, Mehrdad, Farinaz Koushanfar, and Miodrag Potkonjak. "Testing techniques for hardware security." In *Test Conference, 2008. ITC 2008. IEEE International*, pp. 1-10. IEEE, 2008.
- [23] Sedcole, Pete, and Peter YK Cheung. "Within-die delay variability in 90nm FPGAs and beyond." In *Field Programmable Technology, 2006. FPT 2006. IEEE International Conference on*, pp. 97-104. IEEE, 2006.
- [24] Delvaux, Jeroen, Roel Peeters, Dawu Gu, and Ingrid Verbauwhede. "A survey on lightweight entity authentication with strong PUFs." *ACM Computing Surveys (CSUR)* 48, no. 2 (2015): 26.
- [25] Yu, Meng-Day, et al. "A noise bifurcation architecture for linear additive physical functions." *Hardware-Oriented Security and Trust (HOST)*, 2014 IEEE International Symposium on. IEEE, 2014.
- [26] Becker, Georg T. "On the pitfalls of using arbiter-PUFs as building blocks." *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 34.8 (2015): 1295-1307.
- [27] Hansen, Nikolaus. "The CMA evolution strategy: a comparing review." *Towards a new evolutionary computation*. Springer, Berlin, Heidelberg, 2006. 75-102.